0. Machine Learning

Let the machines learn similarly to how humans learn

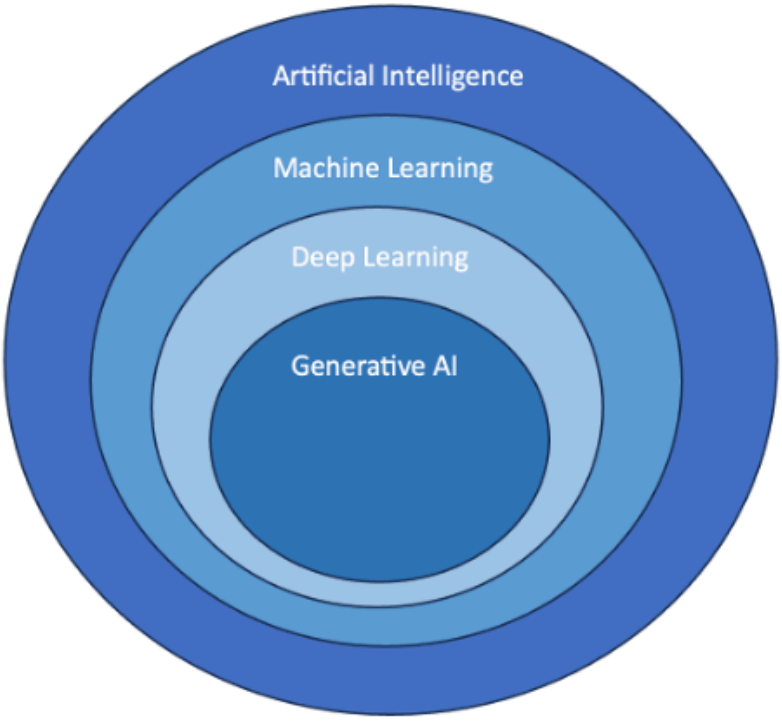
Contents:

Every new subject comes with a first impression. And honestly, *Machine Learning* sounds like one of those terms that tries to explain itself.

"Machines... Learning?" → Seems simple enough, right? → Well, yes—and no.

You've probably heard about *Artificial Intelligence* before. It's everywhere. In news, social media, and tech discussions, people throw the term around all the time. But here's the interesting part: most people who **use** AI don't actually **learn** how it works.

And because of that, they rarely go deeper. Terms like *Machine Learning*, *Natural Language Processing*, or *Deep Learning* just become background noise.



But you're not here to just *use* it. You're here to **understand** it.

So let's slow down for a moment and ask the real question: **What *is* Machine Learning?**

Before we jump into definitions, think about how humans learn. You weren't born knowing how to recognize a cat. You didn't memorize a rule like:

> ***"Two ears + whiskers + tail = cat."***

You just saw cats. Again and again. And eventually, your brain picked up the patterns. → That's the key idea.

Machine Learning tries to do something very similar. Instead of writing strict rules for a computer like:

> "If this happens, do that" → "If $X > Y$, then output Z"

We let the machine **learn patterns from data**. Not by memorizing instructions. But by **observing examples**.

So instead of programming intelligence directly, we do something much more interesting: We give the machine **experience**. And from that experience, it learns. That's the core philosophy behind Machine Learning.

And in the next section, we'll turn that idea into something more precise.

1. Machine Learning – Learning to extract patterns and relationships

Now let's finally answer the question properly:

"What is Machine Learning?" and more importantly: **"What do we actually mean when we say a machine learns?"**

Imagine this: You're trying to teach a child how to recognize animals. You *could* sit there and explain everything in detail:

- ***"A cat has this shape..."***
- ***"A dog has that structure..."***
- ***"If the ears look like this, then..."***

But that quickly becomes messy. There are too many rules, too many exceptions. So instead, you do something much simpler → You show the child **examples**.

Hundreds of them. Cats labeled "cat", dogs labeled "dog".

And over time, the child just *gets it*. → They don't memorize rules. → They **learn patterns**.

Machine Learning works in exactly the same way. Instead of writing step-by-step instructions for every situation, we give the machine **data**—lots of it. These are often called **labeled examples**, where each input comes with the correct answer.

And then, we let the machine figure things out on its own. Not by memorizing, but by **extracting patterns and relationships** hidden inside the data.

So what does that really mean? When a Machine Learning model is learning, it is doing three very important things:

- **Finding patterns:** It looks through massive amounts of data and starts noticing regularities.
    - Just like you might notice that dark clouds usually mean rain, the model begins to detect repeating signals in the data.
- **Understanding relationships:** But it doesn't stop at patterns. It tries to understand how different pieces of information connect with each other.

For example:

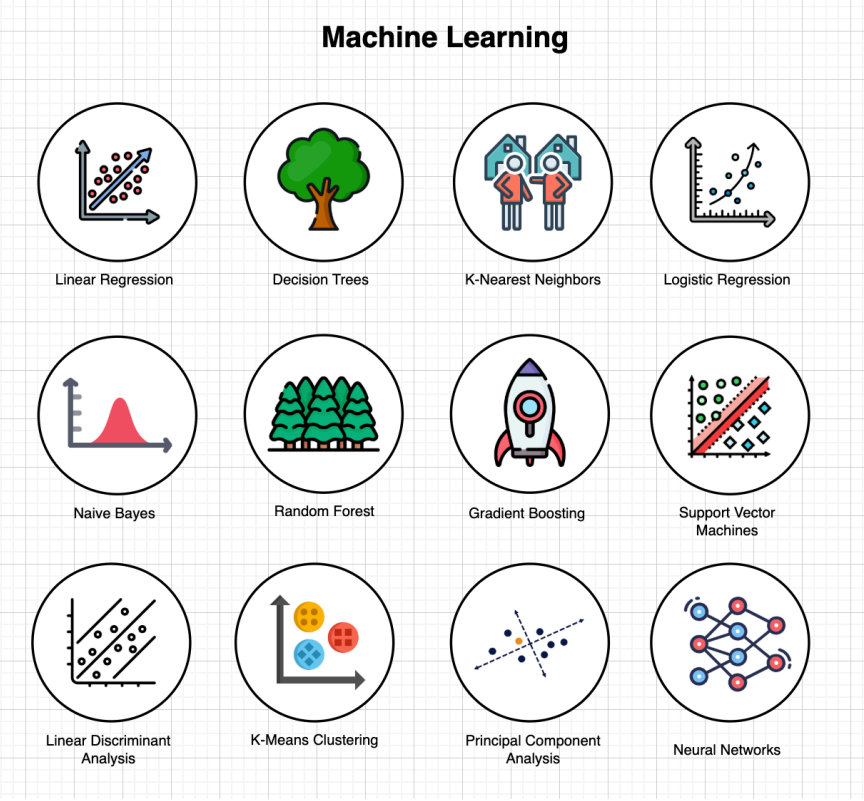
- Certain words often appear together in positive reviews
- Others tend to appear in negative ones
- The model learns these relationships without being explicitly told.

- **Making predictions:** Once it has learned enough, it can face something completely new... and still make a decision.
  - Just like you predicting rain based on experience, the model uses what it learned to **predict unseen data**.

So the goal of a Machine Learning model becomes very clear: It must detect relationships between many given patterns. → Not memorize. Not copy. But **understand structure from data**. → This is what we mean by *learning*.

And here's the important shift. Sometimes, writing rules directly is just impossible. Too many cases. Too many variations. So instead of forcing ourselves to code every rule, we let the machine **discover the rules**.

Different models do this in different ways:



- Some think in **lines and equations** (like Linear models)

- Some think in **decisions and hierarchies** (Tree-based models)
- Some separate data using **boundaries** (Support Vector Machines)
- Some compare using **similarity** (Nearest Neighbors)
- Some simply try to **find hidden groups** (like K-Means)

Each one is like a different *strategy* for learning. And depending on the problem, we choose the one that fits best.

So in the end, Machine Learning is not about giving machines intelligence directly. It's about something much more subtle.

We give them data. We give them experience. And we let them **learn how to understand it**.

2. Machine Learning – Acquiring knowledge from information

Let's take that idea one step further. If Machine Learning is about learning patterns, then what is it *really doing* underneath?

Here's the deeper answer: Machine Learning is the process of turning **raw information into useful knowledge**.

Because let's be honest, raw data, by itself, is kind of meaningless. A table full of numbers doesn't *tell* you anything. It just sits there.

But the moment a model starts analyzing it—finding patterns, detecting relationships—that's when something interesting happens. → **Information becomes knowledge.**

And this transformation doesn't happen instantly. It comes from **repeated exposure**.

The model sees example after example, again and again, and slowly builds an understanding of the problem → The more *good* examples you give it... the better that understanding becomes.

Let's make this real. Imagine we want to build a system that predicts **house prices** → Sounds simple, right? But think about what actually affects the price of a house.

Each house comes with a set of **features**—its characteristics:

- How big it is (square footage)
- How many bedrooms and bathrooms

- Where it’s located
- How old it is
- Whether it has a pool, a garage, or something extra

All of these together form what we call the **training data**.

Now here’s where Machine Learning does its thing. The model looks at thousands of houses, along with their actual prices. And starts asking a silent question:

> *“**What patterns connect these features to the price?”***

Over time, it begins to uncover relationships like:

- Bigger houses tend to cost more
- Certain locations are consistently more expensive
- Newer houses often have higher value (with exceptions)
- Pools matter a lot, but only in the right regions
- More bedrooms increase price, but not forever

Notice something important here. These are not rules we explicitly wrote. These are **relationships the model discovered**.

And now we reach the most important moment: **Prediction.**

Once the model has learned enough, we can give it a house it has never seen before.

For example:

- 2,000 square feet
- 3 bedrooms, 2 bathrooms
- Built 5 years ago
- Has a pool
- Located in a specific area

And the model will do something almost human-like. It will combine *everything it has learned* and estimate the price.

And here’s the beauty of it. The model is not thinking in simple rules like: ***“bigger = more expensive”.*** It’s thinking in **relationships**. It understands that:

- Size depends on location
- Location interacts with features
- Features influence each other

It sees the **bigger picture**.

So when we say: ***"Machine Learning acquires knowledge from information"***. What we really mean is this:

It takes raw data → Finds patterns → Understands relationships → And turns all of that into something powerful → The ability to make decisions about the unknown.

3. Machine Learning – Make Prediction or Memorization?

Here's a question that sounds simple, but is actually *very dangerous* if misunderstood:

> ***"Are we training a Machine Learning model to predict...or to memorize?"***

Because a model *can* memorize data. And when it does, it creates one of the most common problems in Machine Learning: **Overfitting.**

Let me explain it the way I usually think about it. Imagine you're studying for an exam. You have two strategies:

- You **memorize** every question and answer from past papers
- Or you **understand the concepts** behind them

If the real exam gives you the exact same questions, memorization works perfectly. But the moment the questions change—even slightly, you're stuck. Because you didn't actually learn anything. You just remembered.

Machine Learning models can fall into that exact same trap. If a model simply memorizes the training data, it will perform *very well* on what it has already seen... But the moment you give it something new? It fails. Completely.

And that's a problem, because the whole purpose of Machine Learning is this:

> **To make predictions on unseen data.**

Not to replay the past. But to handle the unknown.

Let's go back to our house price example. An overfitted model might do something like this: ***"The blue house at 123 Oak Street costs $350,000".*** → Great, but that's not intelligence. That's memory.

Now give it a new house: **Different street → Similar size → Similar features.** And suddenly, it has no idea what to do. Because it never learned the **relationship** between features and price. It only remembered specific cases.

This is why we say: A good Machine Learning model must **generalize. Here,** Generalization means:

- Learning patterns that apply beyond the training data
- Understanding relationships, not storing examples

And here's something that often surprises beginners: A model that is **100% accurate** is usually a bad sign → **Yes, really.**

Because in many cases, perfect accuracy means the model has memorized the training data— not learned from it.

In reality, what we want is something more balanced. A model that doesn't need to be perfect. But consistently makes **good predictions.** Typically somewhere around **80%–99% accuracy**, depending on the problem.

So the real goal becomes: ***Learn enough to understand the data. But not so much that you memorize it***. It's a balance. Not:

- Too little learning → the model is clueless.
- Too much memorization → the model becomes rigid.

But right in the middle. That's where Machine Learning actually works.

4. Five Tribes of Machine Learning

Let's step back for a moment. Up until now, it might feel like Machine Learning is just *one thing*. One idea. One approach. But that's not really true.

Machine Learning is more like a **collection of philosophies**—different ways of thinking about how machines should learn.

I like to think of them as **five tribes**. Each tribe looks at the same goal—*intelligence*—but approaches it in a completely different way.

- **Symbolists — "Intelligence through logic":** The Symbolists are the oldest thinkers in Machine Learning.

    - They believe intelligence comes from **clear rules and logical reasoning**.
    - Everything is structured. Everything follows a defined path. Think of it like this: ***"If this happens, then do that."*** → Very precise. Very controlled.
    - Back in the early days of AI (around the 1950s–1980s), this was *the* dominant way of building intelligent systems.
    - Examples of their thinking: Decision Trees; Random Forests; Rule-based systems
    - To them, learning is about building a system that can **reason step by step**.
- **Connectionists — "Intelligence through the brain": T**he Connectionists take a completely different path.
    - They look at the **human brain** and say: ***"What if we build something that learns like this?"***
    - Instead of rules, they build **networks**. Layers of neurons → Connections. Signals flowing through the system.
    - And instead of telling the model what to look for, they let it **discover patterns on its own**.
    - This is where things like: Neural Networks; Deep Learning; and Reinforcement Learning come from.
    - If Symbolists are about logic, Connectionists are about **learning through experience and structure**.
- **Bayesians — "Intelligence through probability":** Now this tribe is a bit more subtle. Bayesians don't believe in strict rules. They believe the world is uncertain.
    - So instead of saying: "This is true" or "This is false". They say: *"This is likely... with some probability."*
    - To them, Machine Learning is about **reasoning under uncertainty**. They focus on: Probabilistic models; Statistical inference; and Updating beliefs as new data comes in
    - Examples include: Bayesian Networks; Hidden Markov Models; and Causal models

    - Their mindset is: ***"We don't need certainty... we need confidence."***
- **Evolutionaries — "Intelligence through adaptation".** Now this one is inspired by biology. Evolutionaries look at nature and think: *"Intelligence doesn't just appear... it evolves."*
    - So instead of directly designing a model, they let models **evolve over time**.
    - They mutate → They compete → The best ones survive → Just like natural selection.
    - They use: Genetic Algorithms or Evolutionary Programming
    - You'll often see this in areas like Game AI and Optimization problems. Where systems gradually improve by adapting to their environment.
- **Analogizers — "Intelligence through similarity":** And finally, the Analogizers. These are the storytellers. They believe learning comes from **comparison**.
    - Instead of building complex rules or deep structures, they ask: *"Have I seen something like this before?"* → If yes, then: *"This new thing should behave similarly."*
    - They group data into **classes**, and make predictions based on resemblance. A classic example is the K-Nearest Neighbors (KNN) algorithm
    - Simple idea: Find the most similar cases → Use them to decide → No deep theory. Just **analogy**.

So, what does this mean? Five tribes. Five completely different ways of thinking.

- Logic (Symbolists)
- Brain-inspired learning (Connectionists)
- Probability (Bayesians)
- Evolution (Evolutionaries)
- Similarity (Analogizers)

And here's the important part: "*There is no single 'correct' way".* Each tribe shines in different problems. Each one solves intelligence in its own way.

So when you study Machine Learning, you're not just learning algorithms. You're exploring **different philosophies of learning itself**. And over time, you'll start to see which tribe you think like.

5. Machine Learning Pipeline

Let's talk about something that beginners often misunderstand. They think Machine Learning is just: **"Train a model → Test it → Done."**

But in reality? That's just a **tiny part** of the story. Machine Learning is not a one-time action. It's a **process**. And more importantly, it's an **iterative process**.

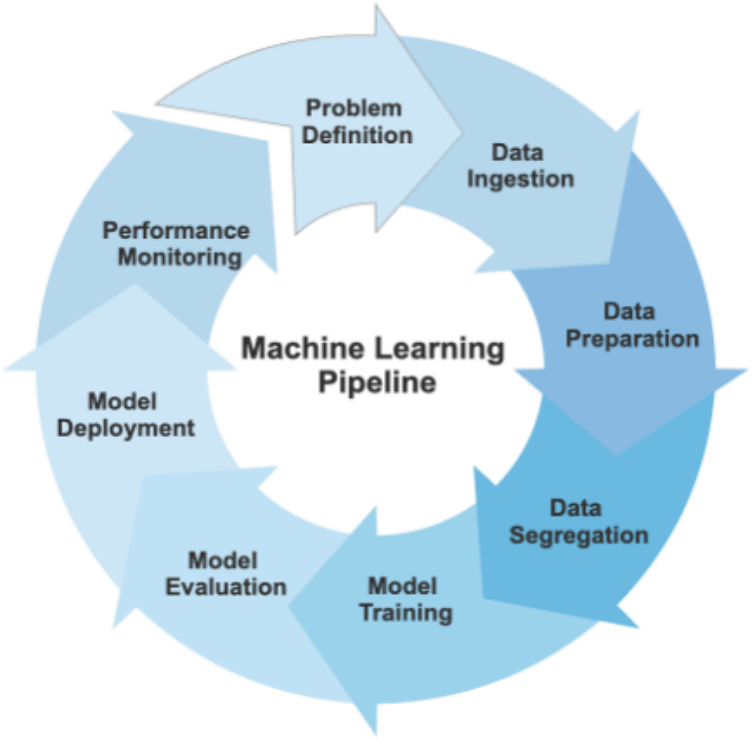
What does that mean? It means we don't just build a model once and walk away.

We: ***Train it → Evaluate it → Improve it → Retrain it → Repeat... again and again***

Because data changes. The world changes. And if the model doesn't adapt, it becomes useless.

So instead of thinking in steps, think in a **cycle**. A loop that keeps refining itself over time.

Let's walk through that cycle (reference the image below).



- **Problem Definition:** Everything starts with a question. What are we trying to solve?
  - Are we predicting house prices?
  - Classifying emails as spam or not?
  - We also define the **target**—the thing we want the model to predict. Without this, nothing else makes sense.
- **Data Ingestion:** Next, we gather data. From files, databases, APIs... wherever it lives. This includes: Features (inputs) and Target (output). No data → no learning.
- **Data Preparation:** Now comes one of the most important (and messy) steps. Raw data is rarely usable. So we clean it. We fix:
  - Missing values
  - Duplicate records
  - Inconsistent formats

    - We might also normalize values; create new features (feature engineering); remove irrelevant or highly correlated data

  This step is where good models are often *made or broken*.

- **Data Segregation:** Before training, we split the data: Training set; Validation set; and Testing set. Why? Because we don't want the model to just memorize. We want to test it on **data it has never seen before**. This is one of the key defenses against overfitting.
- **Model Training:** Now, the actual learning begins. We feed the training data into the model. And it starts:
    - Detecting patterns
    - Learning relationships
    - Adjusting itself

  Sometimes, we even bring in **external data** to make the model more robust.

- **Model Evaluation:** After training, we test it. Like a student taking an exam. We give it unseen data and ask: *"How well did you actually learn?"*
    - We measure: Accuracy; Precision; and other performance metrics

  This tells us whether the model is ready, or needs improvement.

- **Model Deployment:** If the model performs well, we move it into the real world. Now it's no longer just an experiment. It becomes something people can use:
    - Predict prices
    - Detect faces
    - Power applications

  This is where Machine Learning becomes *useful*.

- **Performance Monitoring:** But we're still not done. Even after deployment, we keep watching the model. Because real-world data changes. And over time, the model's performance can **degrade**. So we:
    - Track its predictions
    - Compare them with real outcomes
    - Retrain when necessary

And now, we're back to the beginning. Because this is a loop. ***Output today becomes input tomorrow.***

In this course, we won't focus on everything. We'll mainly focus on: ***Data Preparation → Training → Evaluation***

But it's important to see the **big picture**. Because Machine Learning isn't just about building models. It's about **maintaining them over time**.

6. Python for Machine Learning

Let's talk tools. Because everything we've discussed so far, we don't build from scratch. Thankfully.

When working with Machine Learning in Python, we rely on **libraries**—pre-built packages that do the heavy lifting for us. Instead of writing thousands of lines of complex code, we use tools that already exist.

Here are the core ones you'll see again and again:

- **NumPy** → for numerical computations (arrays, matrices)
- **SciPy** → for scientific and mathematical operations
- **Pandas** → for handling datasets (tables, dataframes)
- **Scikit-learn** → the *main character* (models, training, evaluation)
- **Matplotlib** → for plotting and visualization
- **Seaborn** → for more advanced and beautiful visualizations

Think of it like this: You're not building a machine from raw metal. You're assembling it using powerful, ready-made parts.

And with these tools, everything you've learned about Machine Learning becomes **practical**. Not just theory. But something you can actually build.

Summary

This section gives you everything you need to know about Machine Learning, at least, the *foundation*.

We started with a simple idea: *Machine Learning is about letting machines **learn from data**, instead of explicitly programming every rule. Just like humans learn from experience, models learn by observing examples, detecting patterns, and understanding relationships.*

From there, we went deeper. We saw that Machine Learning is not just about patterns—it is about **transforming information into knowledge**. Raw data means nothing on its own, but once a model processes it, it becomes something powerful: the ability to make predictions about things it has never seen before.

But we also faced an important challenge. A model that simply memorizes data is not intelligent—it is overfitting. The true goal is **generalization**: learning patterns that apply beyond the training data. Not perfect accuracy, but reliable performance on new, unseen inputs.

Then, we explored the idea that Machine Learning is not a single approach, but a collection of **different philosophies**:

- Logic-driven systems
- Brain-inspired networks
- Probabilistic reasoning
- Evolution-based learning
- Similarity-based thinking

Each one offers a unique way to approach intelligence.

After that, we zoomed out and looked at the bigger picture: the **Machine Learning pipeline**. From defining the problem, preparing data, training and evaluating models, to deploying and continuously improving them—Machine Learning is an **iterative process**, not a one-time task.

And finally, we grounded everything in practice with Python—using powerful libraries that allow us to build real models without starting from zero.

So if you take one thing away from this section, let it be this:

> Machine Learning is not about writing smarter code. It's about building systems that can **learn, adapt, and improve over time**

And this? This is just the beginning.

References

https://www.ibm.com/think/topics/machine-learning

https://developers.google.com/machine-learning/intro-to-ml/what-is-ml

https://www.datacamp.com/blog/what-is-machine-learning

https://www.bmc.com/blogs/machine-learning-tribes/

https://www.youtube.com/watch?v=63Kr3HFECHM

Next Section:

🤖 1. Data Preparation for Machine Learning